Comfort Private Hospital Outpatient Database

Project scope

Comfort Private Hospital is a fictional hospital. I designed its outpatient database around two tasks: booking a clinician's appointment slot and retrieving a patient's visit history. Registration staff manage patient records and bookings, clinicians record consultations and diagnoses, and laboratory staff enter test results. The database uses synthetic records from the Synthea sample dataset [1].

I implemented the relational database in PostgreSQL and used it to examine query performance and concurrent bookings. I then modelled outpatient visit histories as MongoDB documents and deployed them to Atlas. The final activity uses the booking system to discuss a proposed PostgreSQL replication and failover design.

Relational design and verification

The schema separates appointments from encounters. An appointment records a scheduled booking, while an encounter records a visit that took place. Keeping them separate allows the hospital to retain a missed appointment in its schedule and import a historical encounter even when its original appointment record is unavailable.

I stored patient and provider details in their own tables and linked encounters to them by ID. To verify the relationship, I loaded one synthetic patient, one clinician, and two outpatient visits. A SQL join returned both visits for the same patient, confirming that a single patient record could support multiple encounter records.

Found in: Entity–relationship diagram · Verification query

Query performance and booking integrity

To test patient-history retrieval, I ran the same query before and after adding a composite index on patient ID and visit start time to a 50,002-row benchmark table. PostgreSQL switched from a sequential scan to a bitmap index scan followed by a bitmap heap scan. Both plans returned the same two visits and used a separate sort. Reported execution time fell from 10.507 ms to 3.015 ms, while planning time rose from 0.184 ms to 61.582 ms. Each condition was measured once, so the result establishes a change in the access plan and records the timings for those two runs [2].

I also used two PostgreSQL sessions to test concurrent bookings for the same clinician's 09:00 slot. The second request used a different appointment ID. After the first booking committed, PostgreSQL rejected the competing insert with SQLSTATE 23505 and identified the partial unique index on clinician and start time. This showed that the database enforced the slot rule for appointments with status booked. In a separate transaction, I inserted a 10:00 booking, rolled it back, and verified that no record remained for that test appointment [3].

Evidence. Before-index plan · After-index plan · Booking test output

Document modelling and cloud design

I converted 855 Synthea outpatient encounters into MongoDB documents and bulk-imported them into Atlas. The conversion matched clinical rows to each visit by encounter ID. Each document embedded the visit's diagnoses, prescriptions, and laboratory results because those details are commonly read together. I retained the patient and provider IDs so encounters could also be retrieved across visits. In Atlas, the patient-history query for the example patient returned five outpatient encounters [1, 4, 5].

For the appointment system, I proposed a PostgreSQL primary and an asynchronous standby. The primary would handle booking writes and availability checks. The standby could serve reports that can tolerate a delay. If a booking has committed on the primary and the standby is still catching up, a report may briefly show that slot as available. The primary would enforce the same unique index tested in the booking exercise. During failover, a recently confirmed booking could be missing if its change had yet to reach the standby. This was a proposed deployment, based on the booking behaviour I tested locally [6, 7].

Evidence. Atlas query result · Import and query evidence · Replication and failover discussion

Individual technical reflection

During the first booking test, I accidentally ran the same insert twice. PostgreSQL reported a primary-key violation, and the transaction rolled back. I repeated the test using a fresh slot and a different appointment ID in each session. This time, the error identified the slot index. Comparing the two errors taught me to check which constraint failed and whether the transaction actually committed before interpreting the result. I applied the same care to the performance exercise by reporting planning and execution times separately.

AI-use declaration

I used OpenAI to assist with drafting code, editing grammar, and summarising my work. I was responsible for the critical reasoning and implementation decisions, and I carried out the hands-on implementation and execution myself. I reviewed the suggested code and checked the results in PostgreSQL and MongoDB Atlas.

References

1. Synthea. Sample data downloads.
2. PostgreSQL 18 Documentation. EXPLAIN.
3. PostgreSQL 18 Documentation. Partial Indexes.
4. MongoDB Documentation. Embedded Data in Your MongoDB Schema.
5. MongoDB Database Tools Documentation. mongoimport.
6. PostgreSQL 18 Documentation. Log-Shipping Standby Servers.
7. PostgreSQL 18 Documentation. Hot Standby.